



第 3 章

Stacks and Queues

栈与队列

内容

- 栈
- 队列
- 循环队列
- 迷宫问题
- 表达式求值

栈

❖定义：是一个有序列表，其中插入（也称为入栈或添加）和删除（也称为出栈或删除）操作都在一端进行，这一端被称为**栈顶**

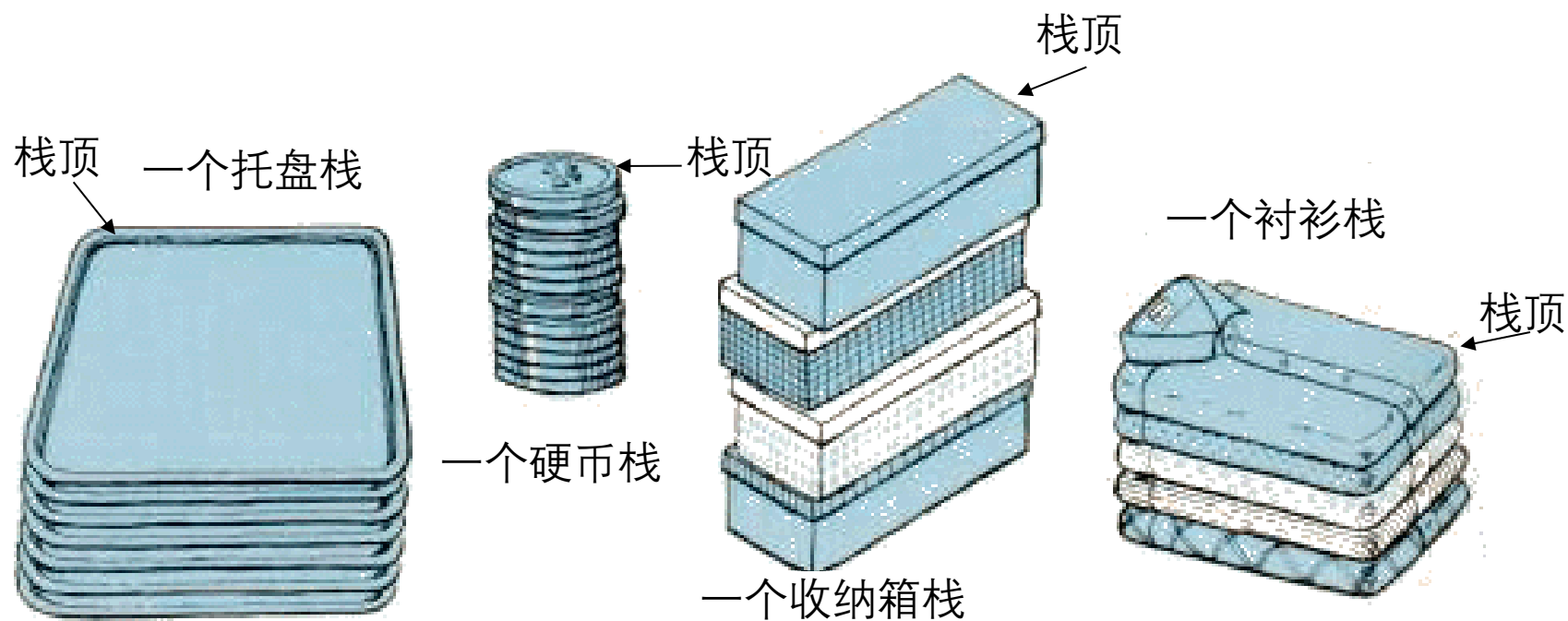
❖定义一个栈 $S=(a_0, \dots, a_{n-1})$

- a_0 是一个栈底元素
- a_{n-1} 是一个栈顶元素
- a_i 位于 a_{i-1} 元素上面的元素，取值范围： $0 < i < n$

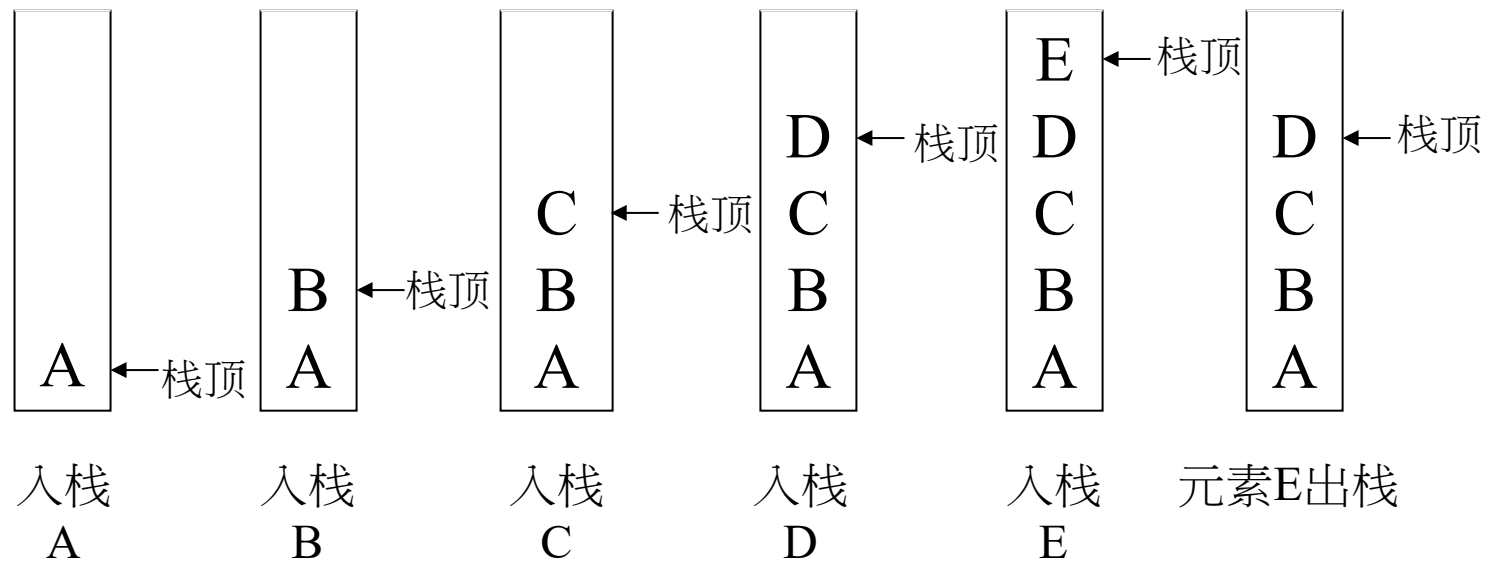
❖后进先出 (Last-In-First-Out: LIFO)

- 新元素从栈顶入栈
- 获取或者删除元素时，只能在栈顶操作

栈示例



栈示例



系统栈

- ❖ 程序运行时用系统栈来管理**函数调用**
- ❖ 每次调用函数时，程序会在系统栈顶部放入一个“**栈帧**”(记录函数信息)
- ❖ 前一个栈（旧）帧指针指向调用函数的栈帧
- ❖ 栈帧里存着函数结束后要返回的代码地址
- ❖ 如果函数再调用另一个函数，当前函数的本地变量（除了静态变量）和参数会加到它的栈帧里

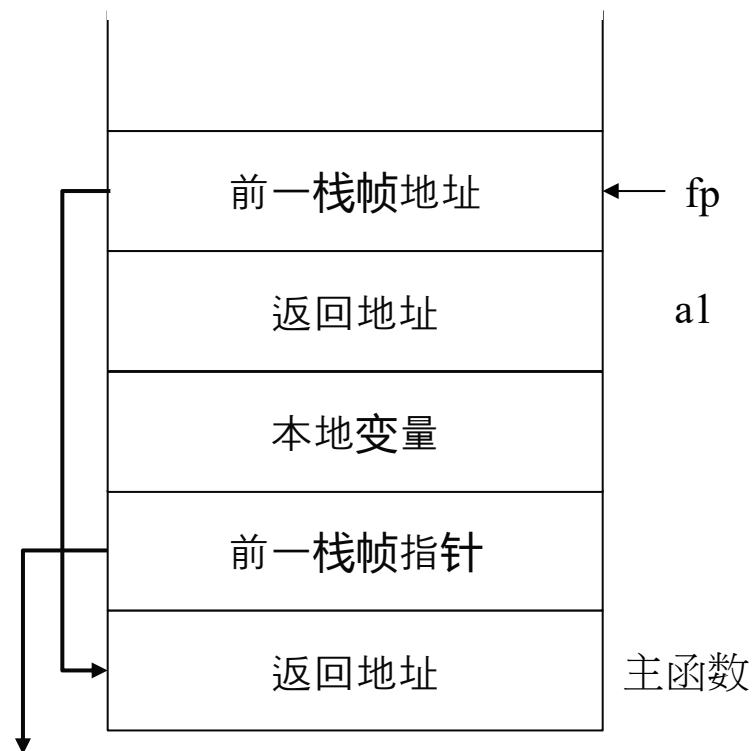
系统栈

❖ 函数调用后的系统栈

- fp: 当前栈帧的指针

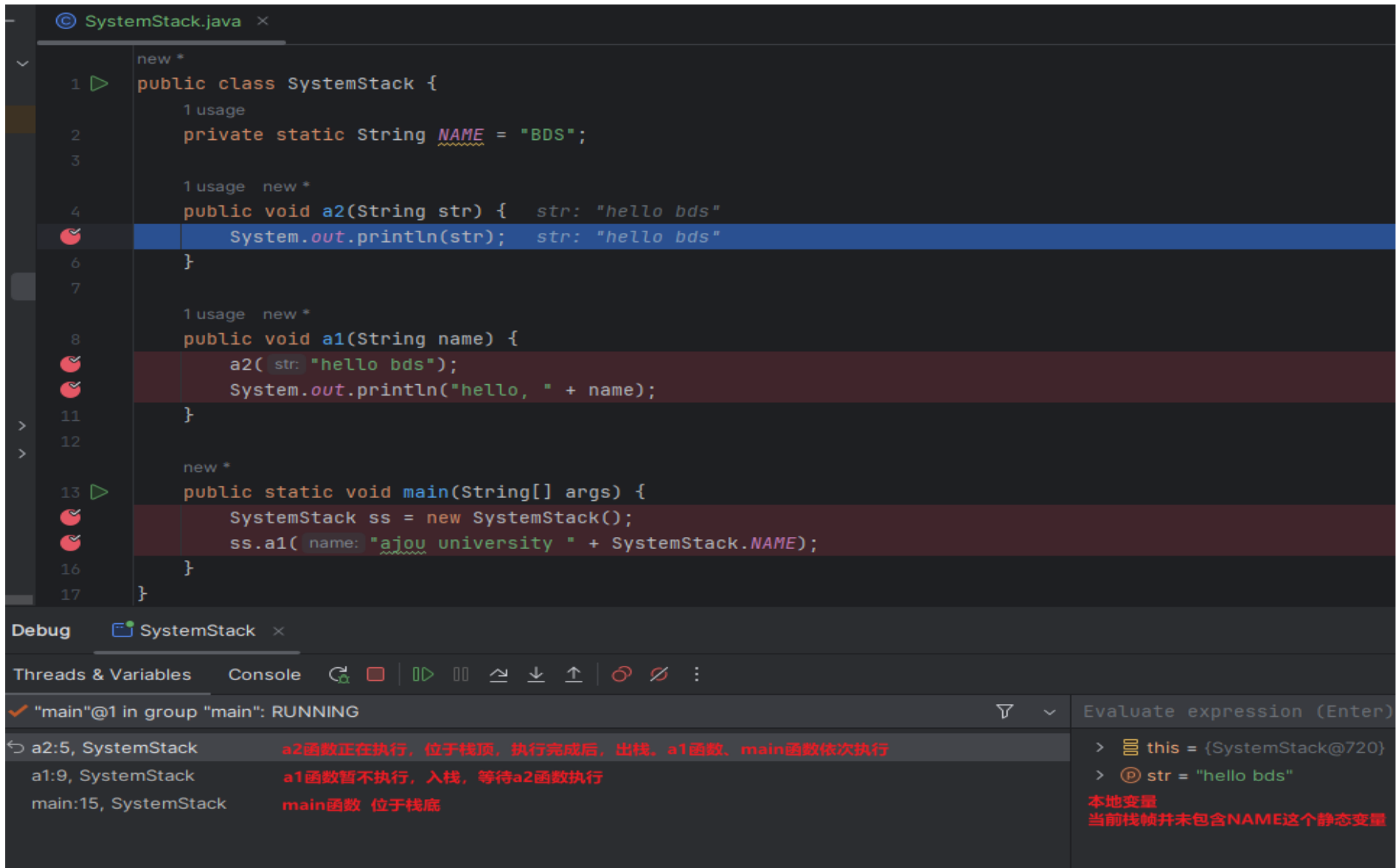


(a) 调用a1(函数)之前的系统栈



(b) 调用a1(函数)之后的系统栈

系统栈



栈抽象数据类型(Stack ADT)

ADT *Stack* is

objects: 一个有限的有序列表, 可以包含零个或多个元素

functions:

for all $stack \in Stack, item \in element, maxStackSize \in \text{positive integer}$

Stack CreateS($maxStackSize$) ::=

建一个最大容量为 $maxStackSize$ 的空栈

Boolean IsFull($stack, maxStackSize$) ::=

if (栈中元素数量 == $maxStackSize$)

return TRUE

else return FALSE

Stack Push($stack, item$) ::=

if (IsFull($stack$)) *stackFull*

else item元素入栈 and **return**

Boolean IsEmpty($stack$) ::=

if ($stack == \text{CreateS}(maxStackSize)$)

return TRUE

else **return** FALSE

Element Pop($stack$) ::=

if(IsEmpty($stack$)) **return**

else 移除并返回栈顶元素

栈的实现

❖使用一维数组 `stack[MAX_STACK_SIZE]`, `MAX_STACK_SIZE`是数组的长度(即栈中最大元素个数)

```
#定义栈元素数量最大为100 /* MAX_STACK_SIZE 100 */
typedef struct {
    int key;
    /* 其他字段(属性) */
} element;
element stack[MAX_STACK_SIZE];
int top = -1; /* top为-1时, 表示当前栈为空栈*/
void push( element item ) {
    /* 添加一个item元素至栈顶*/
    if ( top >= MAX_STACK_SIZE-1 )
        stackFull();
    stack[++top] = item;
}
element pop() {
    /* 移除并返回栈顶元素*/
    if ( top == -1 )
        return stackEmpty(); /* 空栈无法出栈元素*/
    return stack[top--];
}
void stackFull() {
    fprintf( stderr, "已达到栈最大元素数量, 无法添加元素" );
    exit( EXIT_FAILURE );
}
```

案例

```
main()
{
    element e,f;
    top = -1;
    e.key=3;    push(e);
    e.key=2;    push(e);
    f=pop();    top = 0;
    printf ("%d  %d\n", top, f.key);
}
```

栈

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
3	2						

控制台(输出)

0 2

队列

- ❖ 定义: 是一个有序列表, 其插入操作 (也称为添加或出队) 和删除操作 (也称为移除或出队) 分别发生在**不同的两端**
 - 所有**插入**操作都在一端进行, 称为“**队尾**”
 - 所有**删除**操作都在另一端进行, 称为“**队首**”
- ❖ 先进先出(First-In-First-Out :FIFO) 列表: 先入队元素先离开队列
 - 队尾插入元素
 - 只可在队首获取或删除元素

队列应用

❖ 商店排队结账



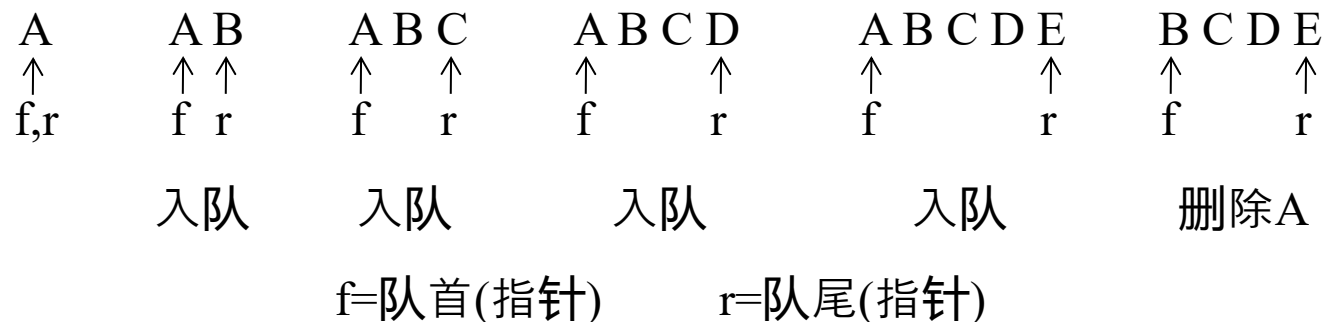
❖ 任务调度

- 常用于计算机编程
- 操作系统中的作业队列
- 任务按照进入系统的顺序进行处理

队列

❖ 队列插入删除元素

- 有5个元素：A, B, C, D, E, 按顺序依次插入队列
- A元素是第一个进入队列，删除时，由于A在队首，因此A会首先被删除



队列抽象数据类型(Queue ADT)

ADT *Queue* is

objects: 一个包含零个或多个元素的有限有序列表.

functions:

for all $queue \in Queue, item \in element, maxQueueSize \in \text{正整数}$

Queue CreateQ($maxQueueSize$) ::=

创建一个空队列, 最大元素数量是 $maxQueueSize$

Boolean IsFullQ($queue, maxQueueSize$) ::=

if (队列中的元素数量 == $maxQueueSize$)

return TRUE

else return FALSE

Queue AddQ($queue, item$) ::=

if (IsFullQ($queue$)) *queueFull*

else 队尾插入元素 $item$ 并返回一个队列(对象)

Boolean IsEmptyQ($queue$) ::=

if ($queue == \text{CreateQ}(maxQueueSize)$)

return TRUE

else return FALSE

Element DeleteQ($queue$) ::=

if (IsEmptyQ($queue$)) **return**

else 移除队首元素并将该元素返回

队列实现

❖使用一个一维数组和两个变量：front 和 rear

```
Queue CreateQ(maxQueueSize) ::=
    #定义队列最大长度MAX_QUEUE_SIZE为100    /* 队列最大长度*/
    typedef struct {
        int key;
        /* 其他字段(属性)*/
    } element;
    element queue[MAX_QUEUE_SIZE];
    int rear = -1;
    int front = -1;

Boolean IsEmpty(queue) ::= front == rear
Boolean IsFullQ(queue) ::= rear == MAX_QUEUE_SIZE-1

void addq( element item ) { /* 队列新增元素item*/
    if ( rear == MAX_QUEUE_SIZE-1 )
        queueFull( );
    queue[++rear] = item;
}

element deleteq() { /* 队首移除元素 */
    if ( front == rear )
        return queueEmpty( );    /* 空队列不可以删除元素*/
    return queue[++front];
}
```


示例

```
main()
{
    element e,f;
    rear = front = -1;
    e.key=3;    addq(e); rear = 0, front = -1
    e.key=2;    addq(e); rear = 1, front = -1
    f=deleteq(); rear = 1, front = 0
    printf("%d %d %d\n", front, rear, f.key);
}
```

队列

[0]	[1]	[2]	[3]	[4]
3	2			

控制台

0	1	3
---	---	---

任务调度

❖ 操作系统创建的任务队列

- 如果操作系统(不对任务)设置优先级，那么任务将按照进入系统的顺序执行

<i>front</i>	<i>rear</i>	<i>Q</i> [0]	<i>Q</i> [1]	<i>Q</i> [2]	<i>Q</i> [3]	Comments
-1	-1					queue is empty
-1	0	J1				Job 1 is added
-1	1	J1	J2			Job 2 is added
-1	2	J1	J2	J3		Job 3 is added
0	2		J2	J3		Job 1 is deleted
1	2			J3		Job 2 is deleted

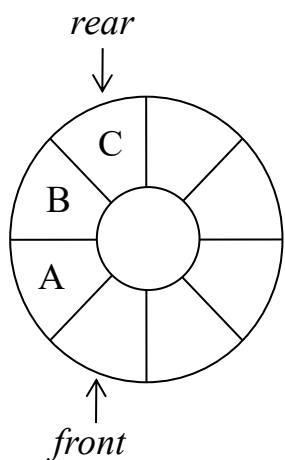
Insertion and deletion from a sequential queue

❖ 随着任务进入和离开系统，队列逐渐向右移动

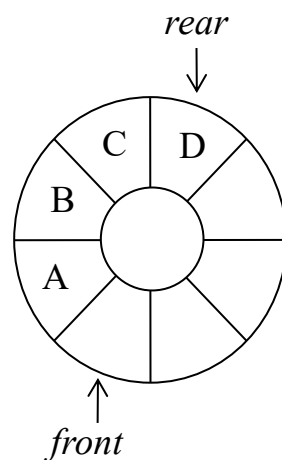
- 队列前面有空闲位置时，应该将整个队列中剩余元素向左移动(如上图，需要将J3移动到Q[0]位置)
- 队列基于数组实现时,由于数组要求在内存中的存储地址连续，因此，删除或移动元素时，需要调整其他元素的位置以保持连续性，会比较耗时。

循环队列 (Circular Queue)

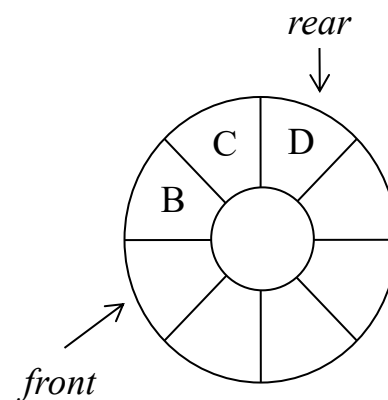
- ❖ 将数组 `queue[MAX_QUEUE_SIZE]` 视为循环队列
- ❖ `front`(队首)索引始终指向队列中队首元素位置的逆时针前一个位置
- ❖ `rear` (队尾)索引指向队列当前的队尾位置
- ❖ 数组最后位置(`MAX_QUEUE_SIZE-1`)的下一个位置是 0，而 0 的前一个位置是 `MAX_QUEUE_SIZE-1`



(a) 初始



(b) 添加元素



(c) 删除元素

循环队列实现

```
void addq( element item )
{
    /* 入队一个元素 */
    rear = (rear+1) % MAX_QUEUE_SIZE;

    /*允许最多容纳 MAX_QUEUE_SIZE-1 个元素，而不是 MAX_QUEUE_SIZE 个元素 */
    if ( front == rear )
        queueFull();    /* 输出错误并结束程序*/
    queue[rear] = item;
}

element deleteq()
{
    /* 删除队列元素 */
    if( front == rear )
        return queueEmpty();
    front = (front+1) % MAX_QUEUE_SIZE;
    return queue[front];
}
```

示例

	queue	[0]	[1]	[2]	[3]		front=0	rear=0
Add 3		[0]	[1]	[2]	[3]		front=0	rear=1
			3					
Add 5		[0]	[1]	[2]	[3]		front=0	rear=2
			3	5				
Add 7		[0]	[1]	[2]	[3]		front=0	rear=3
			3	5	7			
Add 8	错误: 队列已满!!!						front=0	rear=0 rear=3
Delete		[0]	[1]	[2]	[3]		front=1	rear=3
				5	7			
Add 10		[0]	[1]	[2]	[3]		front=1	rear=0
		10		5	7			

迷宫问题

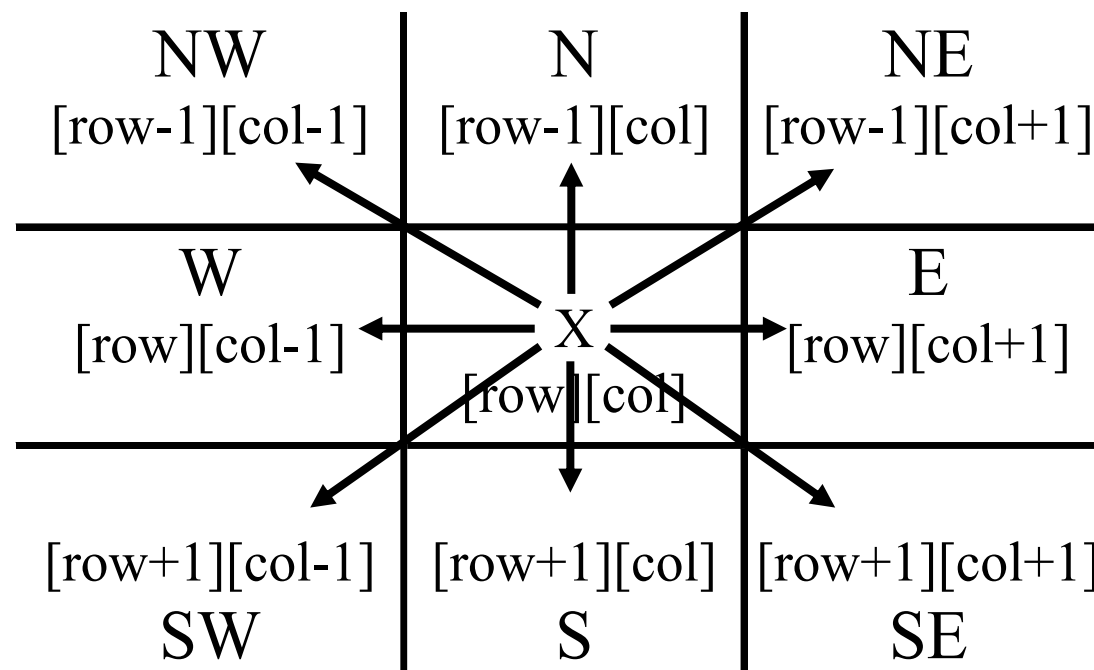
❖ 迷宫

- 最明显的选择:
 - 一个二位数组
 - 0: 代表可走路径
 - 1: 代表障碍物

入口	0	1	0	0	0	1	1	0	0	0	1	1	1	1
	1	0	0	0	1	1	0	1	1	1	0	0	1	1
	0	1	1	0	0	0	0	1	1	1	1	0	0	1
	1	1	0	1	1	1	1	0	1	1	0	1	1	0
	1	1	0	1	0	0	1	0	1	1	1	1	1	1
	0	0	1	1	0	1	1	1	0	1	0	0	1	0
	0	1	1	1	1	0	0	1	1	1	1	1	1	1
	0	0	1	1	0	1	1	0	1	1	1	1	1	0
	1	1	0	0	0	1	1	0	1	1	0	0	0	0
	0	0	1	1	1	1	1	0	0	0	1	1	1	1
	0	1	0	0	1	1	1	1	0	1	1	1	1	0

迷宫问题

- ❖ X 表示当前位置, $maze[row][col]$
 - 可选的移动方向



迷宫问题

❖ 迷宫

➤ 不是所有所有位置都有8个邻居(方向).

- 蓝色区域: 8
- 红色区域: 5
- 黄色区域: 3

0	1	0	0	0	1	1	0	0	0	1	1	1	1	1
1	0	0	0	1	1	0	1	1	1	0	0	1	1	1
0	1	1	0	0	0	0	1	1	1	1	0	0	1	1
1	1	0	1	1	1	1	0	1	1	0	1	1	0	0
1	1	0	1	0	0	1	0	1	1	1	1	1	1	1
0	0	1	1	0	1	1	1	0	1	0	0	1	0	1
0	1	1	1	1	0	0	1	1	1	1	1	1	1	1
0	0	1	1	0	1	1	0	1	1	1	1	1	0	1
1	1	0	0	0	1	1	0	1	1	0	0	0	0	0
0	0	1	1	1	1	1	0	0	0	1	1	1	1	0
0	1	0	0	1	1	1	1	0	1	1	1	1	0	0

迷宫问题

➤ 为避免检查边界条件，用一圈“1”围住迷宫

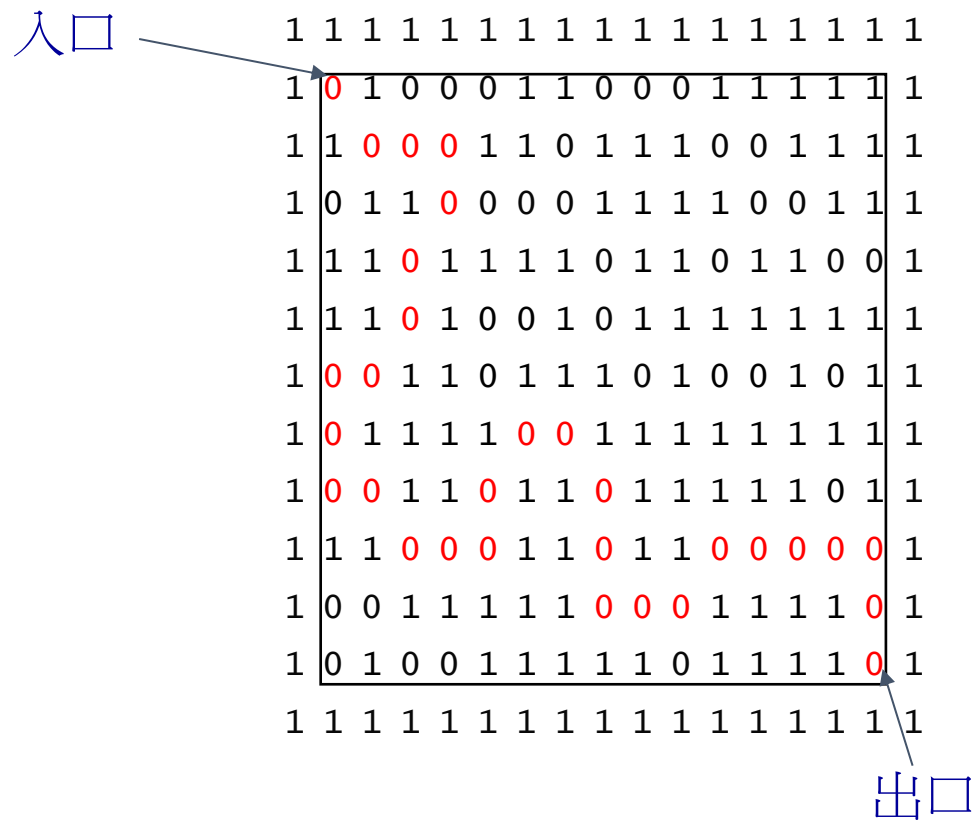
- 一个 $m \times p$ 的迷宫需要使用一个 $(m+2) \times (p+2)$ 的数组来表示
- 迷宫的入口位于位置 $[1][1]$ ，出口位于 $[m][p]$

entrance

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	0	1	0	0	0	1	1	0	0	0	1	1	1	1	1	1	1
1	1	0	0	0	1	1	0	1	1	1	0	0	1	1	1	1	1
1	0	1	1	0	0	0	0	1	1	1	1	0	0	1	1	1	1
1	1	1	0	1	1	1	1	0	1	1	0	1	1	0	0	1	1
1	1	1	0	1	0	0	1	0	1	1	1	1	1	1	1	1	1
1	0	0	1	1	0	1	1	1	0	1	0	0	1	0	1	1	1
1	0	1	1	1	1	0	0	1	1	1	1	1	1	1	1	1	1
1	0	0	1	1	0	1	1	0	1	1	1	1	1	1	0	1	1
1	1	1	0	0	0	1	1	0	1	1	0	0	0	0	0	1	1
1	0	0	1	1	1	1	1	0	0	0	1	1	1	1	1	0	1
1	0	1	0	0	1	1	1	1	1	0	1	1	1	1	1	0	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

exit

迷宫问题



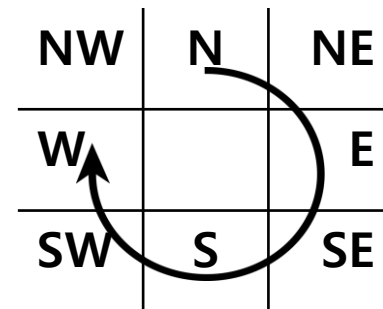
迷宫问题

- ❖ 我们可能面临多个前进方向的选择
- ❖ 在迷宫中，我们可能有多个方向可走
- ❖ 如果发现走错了路，可以通过取出栈顶元素，回到上一个位置并尝试另一个方向

迷宫问题

❖ 实现

```
typedef struct {
    short int vert;
    short int horiz;
} offsets;
offsets move[8]; /*每个方向移动的数组*/
```

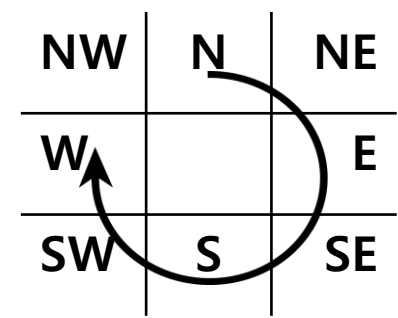


Name	Dir	move[dir].vert	move[dir].horiz
N	0	-1	0
NE	1	-1	1
E	2	0	1
SE	3	1	1
S	4	1	0
SW	5	1	-1
W	6	0	-1
NW	7	-1	-1

❖ 如果我们当前位于迷宫中的位置 `maze[row][col]`，并想找到下一步的位置 `maze[nextRow][nextCol]`，我们需要进行以下设置：

- `nextRow = row + move[dir].vert;`
- `nextCol = col + move[dir].horiz;`

迷宫问题



(row,col)

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)	(0,5)	(0,6)	(0,7)
(1,0)	(1,1)	(1,2)	(1,3)	(1,4)	(1,5)	(1,6)	(1,7)
(2,0)	(2,1)	(2,2)	(2,3)	(2,4)	(2,5)	(2,6)	(2,7)
(3,0)	(3,1)	(3,2)	(3,3)	(3,4)	(3,5)	(3,6)	(3,7)
(4,0)	(4,1)	(4,2)	(4,3)	(4,4)	(4,5)	(4,6)	(4,7)
(5,0)	(5,1)	(5,2)	(5,3)	(5,4)	(5,5)	(5,6)	(5,7)
(6,0)	(6,1)	(6,2)	(6,3)	(6,4)	(6,5)	(6,6)	(6,7)
(7,0)	(7,1)	(7,2)	(7,3)	(7,4)	(7,5)	(7,6)	(7,7)

迷宫问题

❖ 避免重复走过的路径

- 为了不回到之前尝试过的路径，我们维护一个额外的二维数组，用于记录已检查过的迷宫位置
- 将这个数组的所有元素值初始化为 0

❖ 栈的表示

```
#define MAX_STACK_SIZE 100 /*栈的最大容量*/  
typedef struct {  
    short int row;  
    short int col;  
    short int dir;  
} element;  
element stack[MAX_STACK_SIZE];
```

迷宫问题

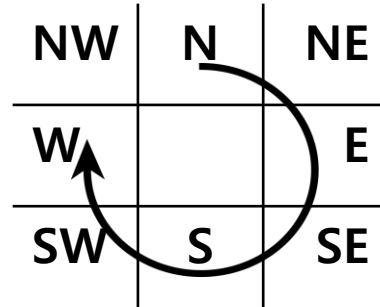
❖ 初始迷宫算法

```
/* 初始化一个栈，使其起始状态包含迷宫入口的坐标，并将初始移动方向设置为北方 */
while ( stack is not empty ) {
    /* 由栈顶获取当前位置和方向 */
    <row, col, dir> = 栈顶元素弹出栈;
    while ( 当前位置存在可走的方向 ) {
        <nextRow, nextCol > = 下一步的坐标;
        dir = 移动方向;
        /* 达到出口，结束 */
        if ( (nextRow == EXIT_ROW) && (nextCol == EXIT_COL) )
            success;

        /* 下一步是否可走 */
        if ( maze[nextRow][nextCol] == 0 &&
            mark[nextRow][nextCol] == 0 ) {
            /* 当前位可走，标记入栈 */
            mark[nextRow][nextCol] = 1;
            /* 保存当前位置和方向,保存至栈顶 */
            row = nextRow;
            col  = nextCol;
            dir  = north;
        }
    }
}

printf("无路可走了\n");
```

	[0]	[1]	[2]	[3]	[4]	[5]	[6]
[0]		1	1	1	1	1	1
[1]		0	0	1	1	0	1
[2]				0	0	1	1
[3]				0	1	1	
[4]					0	1	
[5]						0	
[6]							



ENTRY = (1,1), EXIT = (5,5)

	PUSH (1,1,N)	
	POP	
(1,1)	PUSH (1,1,SE)	mark[1][2]=1
(1,2)	PUSH (1,2,S)	mark[2][3]=1
(2,3)	PUSH (2,3,SE)	mark[2][4]=1
(2,4)	PUSH (2,4,E)	mark[1][5]=1
(1,5)	POP	
(2,4)	PUSH (2,4,W)	mark[3][3]=1
(3,3)	PUSH (3,3,S)	mark[4][4]=1
(4,4)	FOUND!!!	

Maze (Original)

	[0]	[1]	[2]	[3]	[4]	[5]	[6]
[0]	1	1	1	1	1	1	1
[1]	1	0	0	1	1	0	1
[2]	1	1	1	0	0	1	1
[3]	1	1	1	0	1	1	1
[4]	1	1	0	1	0	1	1
[5]	1	0	1	1	0	0	1
[6]	1	1	1	1	1	1	1

stack

[5]	
[4]	(3,3,S)
[3]	(2,4,W)
[2]	(2,3,SE)
[1]	(1,2,S)
[0]	(1,1,SE)

(1,1)(1,2)(2,3)(2,4)(3,3)(4,4)(5,5)

迷宫(路径)查找函数

```
elements stack[MAX_STACK_SIZE];
offset move[8];
int maze[MAX_ROWS][MAX_COLS], mark[MAX_ROWS][MAX_COLS];
int top;

void path(void)
{
    /* 若存在穿过迷宫的路径, 则输出该路径 */

    int i, row, col, nextRow, nextCol, dir, found=FALSE;
    element position;

    mark[1][1]=1; top=0;
    stack[0].row=1; stack[0].col=1; stack[0].dir=1;

    while (top>-1 && !found) {
        position = pop();
        row = position.row;
        col = position.col;
        dir = position.dir;
```

迷宫(路径)查找函数

```
while (dir < 8 && !found) {
    /* 朝dir方向移动 */
    nextRow = row + move[dir].vert;
    nextCol = col + move[dir].horiz;
    if (nextRow==EXIT_ROW && nextCol==EXIT_COL)
        found = TRUE;
    else if ( !maze[nextRow][nextCol] && !mark[nextRow][nextCol]) {
        mark[nextRow][nextCol] = 1;
        position.row = row;
        position.col = col;
        position.dir = ++dir;
        push(position);
        row = nextRow; col = nextCol; dir = 0;
    }
    else ++dir;
} /* while (dir < 8 & !found)
} /* while (top>-1 && !found) */
```

迷宫(路径)查找函数

```
if (found) {  
    printf("路径是:\n");  
    printf("row  col\n");  
    for (i=0; i<=top; i++)  
        printf("%2d%5d\n", stack[i].row, stack[i].col);  
    printf("%2d%5d\n", row, col);  
    printf("%2d%5d\n", EXIT_ROW, EXIT_COL);  
}  
else printf("迷宫无出路\n");  
}
```

表达式(Expressions)

$((rear+1==front) \ || \ ((rear==MAX_QUEUE_SIZE-1) \ \&\& \ !front))$
 $x = a/b - c+d*e - a*c$

❖ 如果检查表达式，我们会注意到它包含：

- 操作符: $==, +, -, ||, \&\&, !$
- 操作数: $rear, front, MAX_QUEUE_SIZE$
- 括号: $()$

表达式求值

$$x = a/b - c + d * e - a * c$$

设定 $a=4$, $b=c=2$, $d=e=3$. 那么 x 的值是什么? 为什么?

- (运算符)优先规则: $*, / > +, -$
- 运算顺序规则(结合律规则): 由左至右
- 可以使用括号改变顺序.

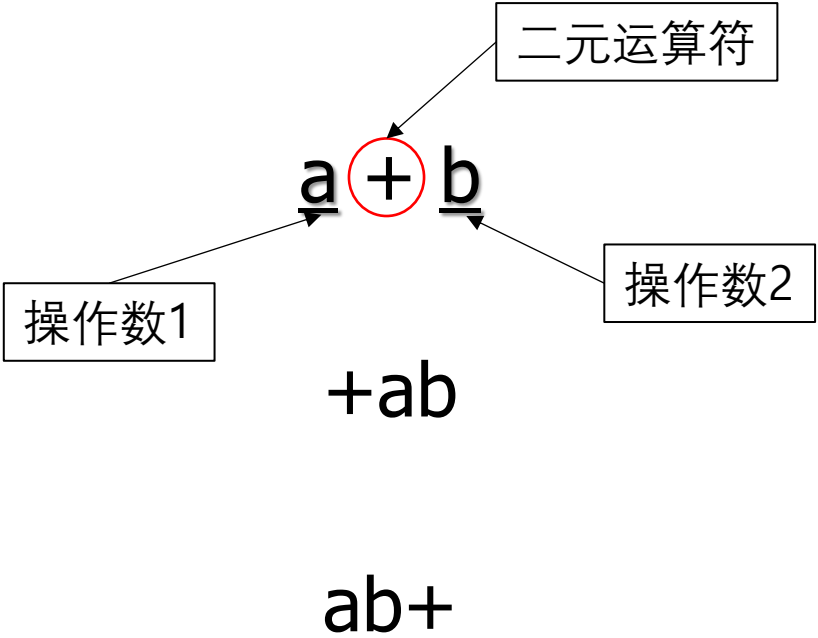
表达式求值

- 使用全括号表达式来消除歧义。
- 每个运算符使用一对括号。

$$x = a/b - c + d * e - a * c$$


$$x = (((((a/b) - c) + (d * e)) - (a * c)))$$

表达式求值



表达式求值

❖ 中缀表示法

- 二元运算符 在两个操作数中间
- e.g. $a+b$, $a-c*d$, $(a+b)*c$

❖ 前缀表示法(波兰表示法)

- 运算符写在操作数之前, 常用于 Lisp 的算术表达式
- e.g. $+ab$, $-a*cd$, $*+abc$

❖ 后缀表示法

- 运算符写在操作数之后, 常被编译器使用
- e.g. $ab+$, $acd*-$, $ab+c*$

表达式求值

中缀表示	前缀表示	后缀表示
$A+B*C$	$+A*BC$	$ABC*+$
$(A+B)*C$	$*+ABC$	$AB+C*$

前缀和后缀表示法无需括号。

中缀表示法

1. 为表达式添加完整的括号，确保每个运算符的操作范围明确
2. 将所有二元运算符移到它们对应的右括号位置。
3. 去掉所有括号。

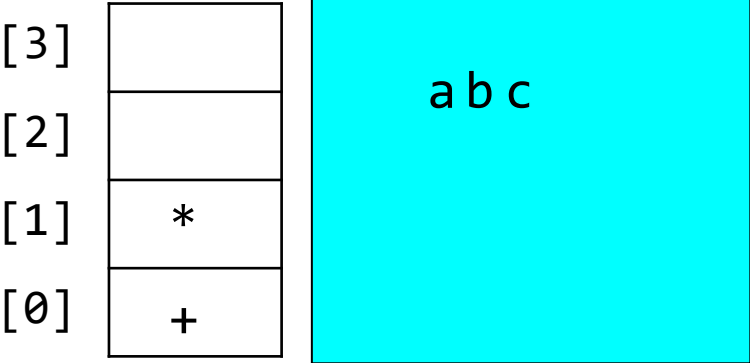
e.g. $A/B - C + D * E - A * C$
→ $((((A/B) - C) + (D * E)) - (A * C))$
→ $(((A/B) - C) + (D * E)) (A * C) -$
→ $((A/B) - C) (D * E) + AC * -$
→ $(A/B) C - DE * + AC * -$
→ $AB/C - DE * + AC * -$

后缀表示法

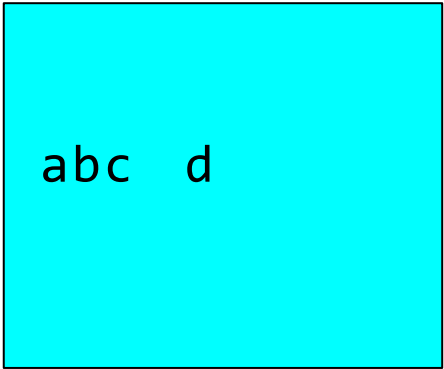
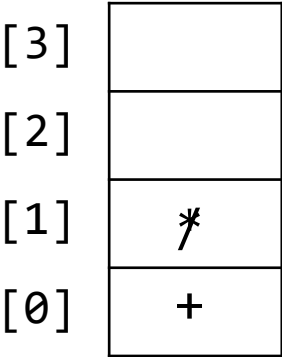
- ❖ 从左到右扫描，逐个检查中缀表达式 (的每个字符)
- ❖ 遇到操作数时，将其传递给输出表达式(中缀和后缀表达式中的操作数顺序相同)
- ❖ 保存运算符，直到我们知道它们的正确位置，并首先输出优先级较高的运算符 只要堆栈顶部运算符的优先级小于传入运算符的优先级，就堆叠运算符
- ❖ 到达表达式末尾时取消堆栈

示例

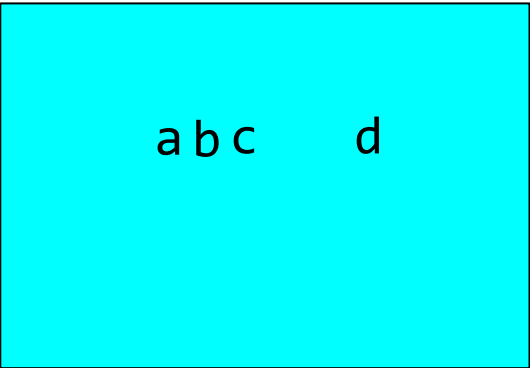
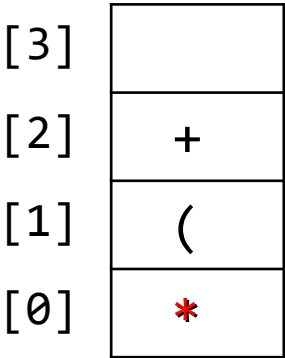
a+b*c



a+b*c/d

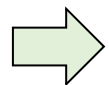


a*(b+c)*d



中缀到后缀

- 遇到左括号(时，将其压入栈中 → **高优先级运算符**
- 在遇到右括号之前，继续将运算符压入栈中暂存
- 左括号只在找到匹配的右括号时出栈 → **低优先级运算符**
- 右括号从不入栈，只用来触发出栈操作



两种优先级类型：

- 栈内优先级in-stack precedence(isp)
- 入栈优先级incoming precedence(icp)

示例

$$a^*(b+c-d)\%e \rightarrow abc+d-*e\%$$
[illegible]

中缀到后缀的实现

```
#define MAX_STACK_SIZE 100  /* 栈最大元素个数 */
#define MAX_EXPR_SIZE 100   /* 表达式最大长度 */
typedef enum {lparen, rparen, plus, minus, times, divide,
              mod, eos, operand } precedence;

int stack[MAX_STACK_SIZE]; /* global stack */
char expr[MAX_EXPR_SIZE]; /* input string */

precedence stack[MAX_STACK_SIZE];
/* isp and icp arrays - 索引表示优先级值
左括号, 右括号, 加号, 减号, 乘号, 出号, 除号, 结束符*/
/* isp: in stack precedence, icp: incoming precedence */
static int isp[] = { 0, 19, 12, 12, 13, 13, 13, 0 };
static int icp[] = { 20, 19, 12, 12, 13, 13, 13, 0 };
```

左括号在输入时具有最高优先级，而在栈内时具有最低优先级。

中缀到后缀的实现

```
void postfix(void)
{
    char symbol; precedence token; int n = 0;
    stack[0] = eos;
    for (token=getToken(&symbol,&n); token!=eos; token=getToken(&symbol,&n)){
        if (token == operand) printf("%c", symbol);
        else if (token == rparen){
            while (stack[top] != lparen) printToken(pop());
            pop(); /* 左括号出栈 */
        }
        else {
            while (isp[stack[top]] >= icp[token]) printToken(pop());
            push(token);
        }
    }
    while ((token=pop()) != eos) printToken(token);
}
```


中缀到后缀的实现

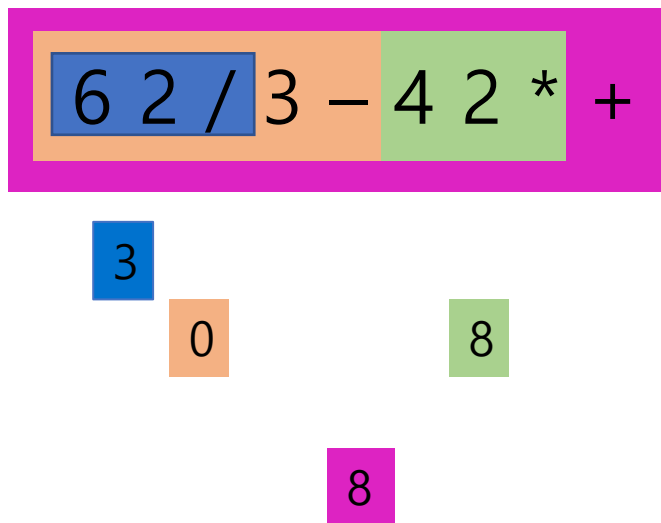
```
precedence getToken (char *symbol, int * n)
{
    *symbol = expr[(*n)++];
    switch (*symbol) {
        case '(' : return lparen;
        case ')' : return rparen;
        case '+' : return plus;
        case '-' : return minus;
        case '/' : return divide;
        case '*' : return times;
        case '%' : return mod;
        case ' ' : return eos;
        default : return operand;
    }
}
```

计算后缀表达式

方便起见，假设表达式中只使用二元运算符和个位整数。

❖ 策略

- 1) 将操作数压入栈中，直至遇到运算符
- 2) 取出两个操作数
- 3) 执行运算，并将结果压入栈中



计算后缀表达式

- ❖ 从左到右扫描后缀字符串
- ❖ 初始化一个空栈
- ❖ 将操作数压入栈中，直到遇到运算符
- ❖ 如果遇到运算符
 1. 从栈中取出该运算符所需的正确数量的操作数
 2. 执行运算
 3. 将结果重新压入栈中
- ❖ 按照此方式继续，直到表达式末尾
- ❖ 然后，从栈顶取出最终答案

示例

- 假定:
- 1) 只有二元运算符
 - 2) 个位整数

62/3-42*+

Token	Stack			Top
	[0]	[1]	[2]	
6	6			0
2	6	2		1
/	3			0
3	3	3		1
-	0			0
4	0	4		1
2	0	4	2	2
*	0	8		1
+	8			0

计算后缀表达式

```
int eval(void){
    precedence token;  char symbol;  int op1,op2;
    int n = 0; top = -1;
    token = getToken(&symbol, &n);
    while (token != eos) {
        if (token==operand) push(symbol-'0');
        else {
            op2=pop(); op1=pop();
            switch(token) {
                case plus:  push(op1+op2);          break;
                case minus: push(op1-op2);          break;
                case times:  push(op1*op2);          break;
                case divide: push(op1/op2);          break;
                case mod:    push(op1%op2);
            } /* switch */
        } /* else */
        token = getToken(&symbol, &n);
    } /* while */
    return pop(); /* return result */
}
```

62/3-42*+

→ 4*2=8

stack

[3]	
[2]	2
[1]	4
[0]	6

```
int eval(void){
    precedence token;  char symbol;
    int op1,op2;
    int n = 0; top = -1;
    token = getToken(&symbol, &n);
    while (token != eos) {
        if (token==operand) push(symbol-'0');
        else {
            op2=pop(); op1=pop();
            switch(token) {
                case plus:  push(op1+op2); break;
                case minus: push(op1-op2); break;
                case times:  push(op1*op2); break;
                case divide: push(op1/op2); break;
                case mod:    push(op1%op2);
            } /* switch */
        } /* else */
        token = getToken(&symbol, &n);
    } /* while */
    return pop(); /* return result */
}
```

```
int stack [MAX_STACK_SIZE];
/* 全局变量 */
```

下次课内容

链表